Deep Learning for Human Activity Recognition: A Performance Evaluation on a ResNet Architecture

Emilio Risi-2122841 Davide Pimazzoni-2160381

***Abstract*—Deep Residual Networks (ResNet) [1] effectively resolve the degradation problem inherent to deep architectures via residual skip connections. In Human Activity Recognition (HAR) systems, 1D-ResNet adaptations [2] process multivariate, continuous time-series signals from wearable inertial measurement units, replacing manual feature engineering with hierarchical feature extraction. To capture spatial-temporal dynamics across dynamic activity sets, hybrid paradigms combine multiscale parallel convolutions with residual connections [3]. This framework reliably preserve gradient flow, optimize convergence, and enhance multi-class classification accuracy across complex human motion sequences.**

In this work, we rigorously evaluate a spectrum of deep learning architectures—namely a Micro-ResNet (ResNet-8), a Multi-scale Inception network, and a Transformer-Encoder—across two distinct classification tasks: Activity Recognition and Contrastive Person Identification. The classification is performed on a dataset of high-resolution Doppler spectrogram traces acquired using commercial IEEE 802.11 (WiFi) devices [4], [5]. These traces correspond to activities such as walking, running, jumping, and sitting. Crucially, while convolutional architectures demonstrate robust capability in extracting local temporal-frequency features, we observe that the self-attention mechanism of the Transformer struggles significantly. Because Doppler spectrograms are inherently highly sparse, the lack of an inductive locality bias forces the Transformer to expend computational resources attempting to correlate empty feature spaces, leading to structural overfitting and poor generalization on unseen environments.

***Index Terms*—Resnet architecture, Human Activity Recognition, Classification task, SHARP, Doppler traces, IEEE 802.11ac standard**

I. Introduction

In recent years, the rapid growth of the Internet of Things (IoT) has driven the development of innovative wireless systems and methodologies. Within healthcare, these wireless devices play a crucial role in monitoring vital parameters [6]. A key research focus is Human Activity Recognition (HAR), which leverages the Doppler shifts in Wi-Fi signals to detect human presence and movement. Implementing these systems is particularly valuable for assisted-living applications in healthcare environments [4].

Since the HAR is a classification problem, a deep-learning architecture like a ResNet [1] can be useful for this kind of task. ResNet addresses the optimization challenges of deep neural topologies by incorporating identity shortcut connections that mitigate vanishing gradients during backpropagation. In Human Activity Recognition (HAR) domains, standard feed-forward architectures often struggle with raw sensor signals. The deployment of modified 1D-ResNet frameworks [2] effectively replaces complex manual feature engineering with hierarchical, end-to-end spatial-temporal feature extraction directly from multivariate time-series data. Furthermore, academic literature highlights the efficacy of integrating residual topologies with recurrent architectures, such as Bidirectional Long Short-Term Memory (BiLSTM) modules [7], or multiscale filter paradigms, such as Inception blocks [3]. These hybrid models seamlessly capture long-range temporal dependencies and fine-grained movement dynamics across complex action sets. Consequently, ResNet-based paradigms ensure stable convergence, enhance classification fidelity, and provide robust generalization.

In this report, we evaluate the performance of three distinct deep learning architectures—a Micro-ResNet, a Multi-scale Inception network, and a Transformer-Encoder—on the dataset described in [4] and [5]. We expand the scope of the analysis beyond standard Human Activity Recognition (classifying activities such as walking (W), running (R), jumping (J), sitting (L), and empty room (E)) by also formulating a Contrastive Person Identification task to differentiate between specific subjects. Through extensive evaluation using loss metrics, accuracy, and confusion matrices, we demonstrate that while convolutional architectures (ResNet and Inception) excel at extracting robust representations from Doppler traces, the Transformer-Encoder fails to generalize. Specifically, Doppler spectrograms are highly sparse representations; without the strong inductive spatial locality bias inherent to convolutions, the Transformer's self-attention mechanism expends capacity attempting to correlate empty space, leading to severe structural overfitting and a collapse in performance on unseen physical environments.

The structure of the rest of the report is the following one:

1) **Related work:** in Section II we describe the main literature regarding ResNet and Inception architectures and their applications in HAR systems;
2) **Processing pipeline & Data:** in Section III and IV we describe how the raw data are sanitized, augmented, and processed for the two respective classification tasks;
3) **Learning framework:** in Section V we detail the structure of the ResNet, Inception, and Transformer architectures;
4) **Results:** in Section VI we show the results of the analysis across all models and tasks.

The work was divided in the following way: both the

Department of Information Engineering, University of Padova, email: emilio.risi@studenti.unipd.it

Department of Information Engineering, University of Padova, email: davide.pimazzoni@studenti.unipd.it

authors of this report defined the architectures of both neural networks and the processing pipeline. Davide Pimazzoni was responsible for finding articles regarding the architectures under examination; Emilio Risi handled the data collection and dataset creation.

II. Related Work

This section contextualizes our approach within the existing literature. Specifically, the following subsections review the state of the art in WiFi-based activity recognition and explore how Resnet has been adapted to these type of task.

A. HAR Systems

In [4], the authors propose SHARP, a framework designed to perform the HAR task using commercial IEEE 802.11 (Wi-Fi) devices. This approach demonstrates that fine-grained human actions can be identified with high accuracy, even when performed by different participants within a controlled environment. Architecturally, the SHARP model relies on a sequence of stacked convolutional layers optimized for localized feature extraction to execute classification. While effective, convolutional neural networks are fundamentally constrained by their localized receptive fields, which limit their capacity to model long-range temporal correlations. To overcome this limitation, our work departs from traditional convolutional topologies in favor of a Resnet architecture; the deep layer topology of ResNet allows the network to learn hierarchically abstract spatial and temporal features automatically directly from raw data, overcoming the limitations and overlap of handcrafted features.

B. Resnet-architecture applied to HAR Systems

About the Resnet architecture and its application on HAR systems, the literature looks confident for this kind of architecture; for example [8] shows that a Resnet suit well on HAR task rather than a simple CNN. Moreover the authors of [8] focus on the fact that by increasing the depth of the network the performances are improved in terms of precision and recall. Based on these considerations, we have implemented a Resnet, according to the architecture proposed by [1].

III. Processing Pipeline

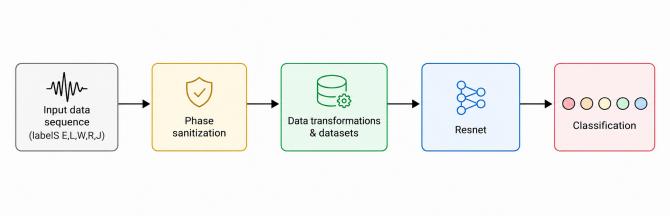


Fig. 1: Processing pipeline.

In this section, we're going to explain the structure of the pipeline, designed for the analysis and processing of the input doppler traces; a sketch of the pipeline is depicted in the flow diagram in Figure 1. In the following, all the processing steps:

1) **Input data:** The raw Doppler traces, associated with specific activities (defined by the labels E, L, W, R, J), serve as the input. These traces are acquired using the techniques proposed by [4] and [5], as detailed in Section IV;
2) **Phase sanitization and computing the Doppler Traces:** This step, using the techniques specified in Section IV, filters environmental and hardware noise from the raw data to ensure its reliability for subsequent analysis. After that, the input data is computed and the data sequence is obtained, as described in Section IV;
3) **Dataset transformation and creation:** Following the cleaning process, transformations are applied to the data to enhance the robustness of the model against variations in the input. The specific techniques applied in this step are detailed in Section IV;
4) **Deep neural architecture and Classification:** The transformed data are processed by a Resnet (whose architecture is defined in Section V) to predict the most probable class associated with the input. The system then outputs a loss value (logits) based on the predict.

IV. Signals and Features

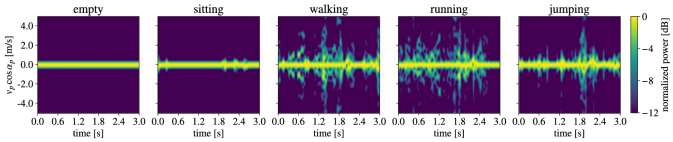


Fig. 2: Spectograms of the five different activities and their associated labels, as proposed by [4]

As described by [5] and [4], the dataset comprises WiFi Channel Frequency Response (CFR) data collected within an IEEE 802.11ac network. The signals were captured by a monitor node (an ASUS RT-AC86U router) while two terminals exchanged traffic on channel 42 operating at a center frequency of 5.21 GHz with an 80 MHz bandwidth while a human subject acted as a physical obstacle to the transmission by performing various activities. Due to constraints in computational resources, this experiment utilizes a subset of the labeled activities and subjects. Specifically, the evaluated activities are empty 'E', sitting 'L', walking 'W', running 'R', and jumping 'J' (Figure 2).

The dataset treatment differs significantly depending on the classification task:

- **Activity Recognition:** The dataset is split such that data from four subjects are allocated to the training and validation phases, while data from two completely distinct, unseen subjects (Subjects 6 and 7) are reserved exclusively for testing to evaluate true model generalization on unseen individuals.
- **Person Identification:** The problem is framed as a binary classification between two specific subjects (Subjects 4 and 5). The test set consists of disjoint tracking

sessions from these same subjects to evaluate identification accuracy on new physical environment sessions rather than zero-shot unseen people. To robustly learn a highly discriminative latent space, the training dataset is dynamically wrapped into a Triplet configuration that generates Anchor, Positive, and Negative samples on the fly. Furthermore, a Weighted Random Sampler is applied during batch generation to perfectly balance the class distribution and stabilize contrastive learning.

About the computation of CFR, in [4], the authors mathematically formulate the raw CFR acquired from commercial Wi-Fi devices communicating via Orthogonal Frequency Division Multiplexing (OFDM). The multi-path CFR across subchannels is defined as:

$$H_k(n) = A_k(n)e^{j\phi_k(n)} = \sum_{p=0}^{P-1} A_p(n)e^{-j2\pi(f_c+k/T)\tau_p(n)} \tag{1}$$

where $H_k(n)$ is the sub-channel response, $A_k(n)$ is attenuation, $\phi_k(n)$ is phase shift, P is the path count, $A_p(n)$ is path attenuation, f_c is carrier frequency, T is OFDM symbol time, and $\tau_p(n)$ is path delay. This raw signal model contains non-ideal hardware phase artifacts that distort real-world measurements.

After that, there's a Sanitization phase, in which this raw CFR is processesed by removing the phase offset, outputting a cleaned signal $\hat{H}_k$, which is computed as:

$$\hat{H}_k \approx A_{p^*}e^{-j2\pi(f_c+k/T)\tau_{p^*}}H_k \tag{2}$$

where $\hat{H}_k$ is the corrected CFR for sub-channel k, A_{p^*} and τ_{p^*} are the amplitude and delay of the strongest reference path p^*, while H_k represents the isolated dynamic multi-path component required for detecting human activity.

Finally the last step of the preprocessing phase is the Doppler Trace Computation, in which the sanitized CFR ($\hat{H}_k$) previously computed are transformed into Doppler spectrograms to isolate human movement velocity from static reflections. By arranging sanitized phase data into timesequenced observation matrices and applying a short-time Fourier transform (STFT), temporal variations are converted into a Doppler spectrogram. The physical movement model is defined as:

$$v_p\cos(\alpha_p) = \frac{uc}{f_cT_cN_D} \tag{3}$$

where $v_p\cos(\alpha_p)$ is the directional velocity of scatterer p, u is the discrete Doppler bin index associated with physical motion, c is the speed of light, f_c is the central carrier frequency, T_c is the sampling period between packets and N_D is the Number of zero-padded signal samples. Specifically, the computation of Doppler traces is performed with the following parameters:

- Number of channels estimates for Doppler computation per sample: 31
- Number of packets for sliding operations: 1
- Number of Doppler vectors per window: 340
- Number of samples for window sliding: 30
- Number of monitoring antenna: 4

Once the sanitized data are obtained, an **Instance Normalization** step is applied: every sample is independently normalized to zero mean and unit variance. This establishes crucial robustness against varying absolute signal reflections across different individuals (particularly vital for the unseen Subjects 6 and 7).

Following normalization, the following transformations are applied sequentially to the training dataset, in order to improve model generalization:

1) **Gaussian noise:** Zero-mean additive white Gaussian noise ($\mu = 0$, $\sqrt{\sigma^2} = 0.1$) is injected into the data sequence to simulate signal fluctuations.
2) **Uniform scale variation:** A scaling factor chosen uniformly from $[0.8, 1.2]$ is applied to randomly alter the apparent duration of the sequence.
3) **Block masking:** A contiguous block representing 10% (proportional ratio 0.1) of the total sequence length is randomly masked in both the time and frequency domains (similar to Cutout or SpecAugment) to train the model to handle missing data.

To ensure unbiased evaluation, these augmentations are disabled during the validation and testing phases. Specifically, the Gaussian noise and block masking parameters are set to 0, and the scale variation factor is fixed to 1.0. These transformations are applied dynamically upon dataset instantiation directly in the GPU VRAM.

V. Learning Framework

We investigate three distinct models for processing the high-resolution Doppler spectrograms, focusing heavily on ResNet architectures.

A. ResNet Architecture

In this section there are detailed descriptions of each block of the Resnet architecture adopted for the project. The architecture of the resnet is developed in the following way:

1) **Input Preprocessing and Channel Injection Block:** The raw input batch tensor $x \in \mathbb{R}^{B\times H\times W}$ (where B denotes batch size, H represents time sequences, and W represents feature dimensions) enters the forward pass and undergoes spatial reshaping via `x.unsqueeze(1)`. This transforms the tensor into a single-channel 2D representation $x \in \mathbb{R}^{B\times 1\times H\times W}$, enabling 2D spatial convolution operators to ingest Doppler spectrograms directly;
2) **Convolutional Stem Block:** The single-channel tensor x passes sequentially through `conv1`, `bn1`, `relu`, and `maxpool`:
   - `conv1` (7×7 kernel, stride $s = 2$, padding $p = 3$): Downsamples spatial dimensions while projecting 1 input channel to C_{base} channels:

$$x_{conv1} = W_{stem} * x \tag{4}$$

- `bn1`: Standardizes activations per channel across the mini-batch to reduce internal covariate shift:

$$\hat{x} = \frac{x_{conv1} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_{bn1} = \gamma \hat{x} + \beta \tag{5}$$

- `relu`: Applies element-wise rectification $\text{ReLU}(y) = \max(0, y)$;
- `maxpool` (3×3 pool, stride $s = 2$): Reduces spatial dimensions by half.

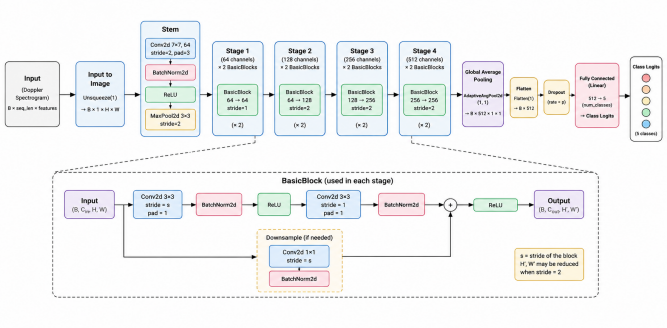


Fig. 3: Resnet architecture

3) **Residual Deep Feature Extraction Stages:** The tensor flows through sequential residual stages where spatial dimensions shrink by factor 2 while channel depth doubles ($C_{out} = C_{in} \cdot 2$). Within each block:
   - **Residual Branch Processing:** Input x_{block} passes through two 3×3 convolutions with intermediate batch normalization and ReLU activation:

$$\mathcal{F}(x_{block}) = \text{BN}_2(W_2 * \text{ReLU}(\text{BN}_1(W_1 * x_{block}))) \tag{6}$$

   - **Shortcut Projection:** The input is saved as $h(x_{block}) = x_{block}$. If dimensions change, a projection adjusts dimensions via 1×1 convolution: $h(x_{block}) = \text{BN}_{ds}(W_{ds} * x_{block})$;
   - **Residual Addition and Regularization:** The shortcut is added element-wise before non-linear activation:

$$y_{block} = \text{ReLU}(\mathcal{F}(x_{block}) + h(x_{block})) \tag{7}$$

   Stochastic feature dropout zeroes activations with probability p = dropout_rate to prevent co-adaptation;
4) **Global Temporal Aggregation and Linear Head:** Spatial dimensions of feature maps are collapsed into a scalar per channel by Adaptive Average Pooling:

$$x_{pool}(c) = \frac{1}{H' \times W'} \sum_{i=1}^{H'} \sum_{j=1}^{W'} x(c, i, j) \tag{8}$$

   The resulting tensor is flattened and projected to category logits z via a fully connected layer.
5) **Weight Initialization:** Initial weights of convolutional and linear layers are initialized using Xavier uniform initialization to bound layer output variances.

B. Inception Baseline Architecture

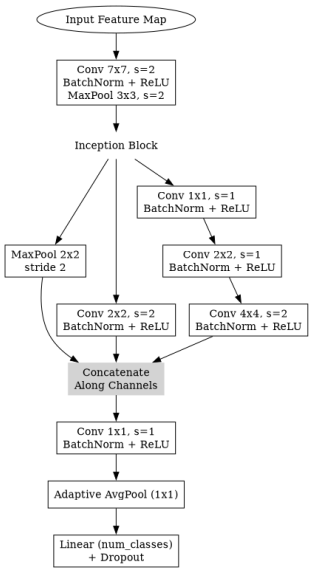


Fig. 4: Simplified Multi-Scale Inception architecture baseline.

To provide a robust multiscale baseline, we additionally developed a simplified SHARP Inception network (Figure 4). This model diverges from sequential deep feature extraction by leveraging parallel convolutional pathways of varying receptive fields within the same layer. The core Inception Module splits the incoming feature map into three distinct branches:

- A structural downsampling branch utilizing a 2×2 Max Pooling operation with stride 2.
- A localized feature extraction branch applying a standard 2×2 convolution with stride 2.
- A deep contextual cascade comprising a 1×1 dimension-reduction convolution, followed by a 2×2 and a wider 4×4 convolution, expanding the receptive field to capture broader temporal-frequency relationships.

The outputs from these parallel branches are then spatially interpolated (to correct sub-pixel rounding discrepancies) and concatenated along the channel dimension. A final 1×1 convolution rapidly reduces the channel depth before propagating to the subsequent block.

C. Transformer-Encoder Architecture [1]

As depicted in Figure 5, the input sequence is initially processed by a Convolutional STEM layer to capture local spatial-temporal patterns and downsample the sequence length by a factor of four, easing the $O(n^2)$ computational complexity of subsequent attention steps. This STEM structure consists of two 2D convolutional layers with batch normalization and Gaussian Error Linear Unit (GELU) activations:

$$\text{Gelu}(x) = x \cdot \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right]$$

where $\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2) dt$. The first convolution accepts the Doppler spectrogram $\mathbf{x} \in \mathbb{R}^{1 \times F \times T}$ using parameters $K = 5 \times 5, P = 2 \times 2, S = 2 \times 2$, projecting it

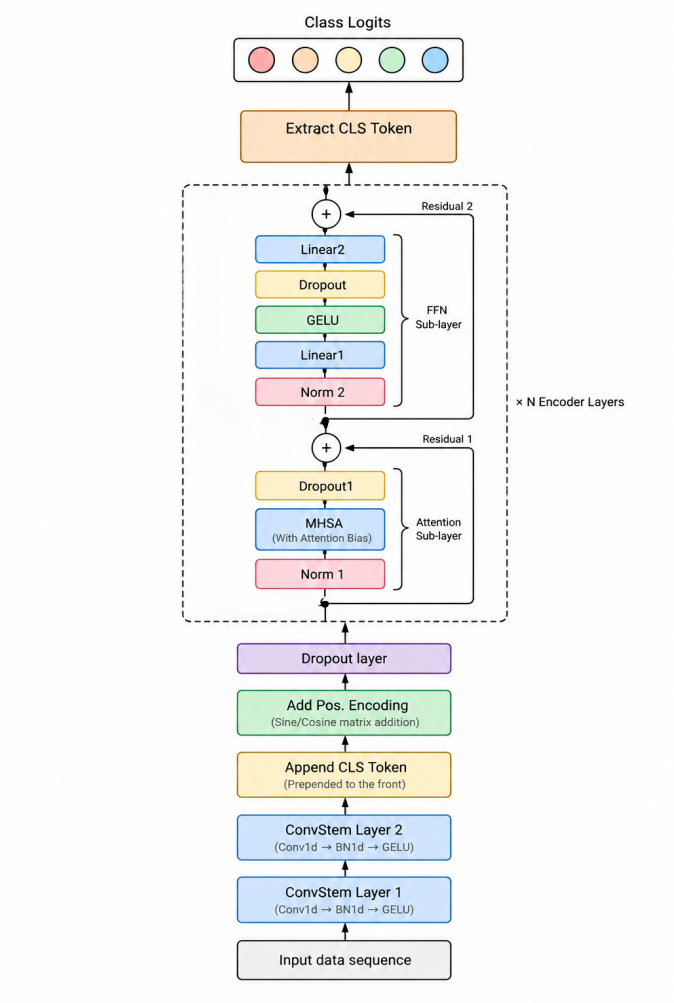


Fig. 5: Transformer-Encoder architecture.

to 16 channels. The second layer operates with parameters $K = 3 \times 3, P = 1 \times 1, S = 2 \times 2$, expanding to 32 channels. The spatial frequency dimension is then flattened and projected to the Transformer's d_{model} dimension. After the STEM processing, a learnable **[CLS]** token is concatenated to aggregate global information for classification. Next, positional temporal information is injected using an alternating matrix $\mathbf{X}$ of size $J \times M$:

$$\mathbf{X}_{(k,2i)} = \sin\left(\frac{k}{10000^{\frac{2i}{M}}}\right), \quad \mathbf{X}_{(k,2i+1)} = \cos\left(\frac{k}{10000^{\frac{2i}{M}}}\right)$$

where k is the token position index ($k = 0, \ldots, J-1$) and i is the dimension index ($i = 0, \ldots, \frac{M}{2} - 1$). The enriched tensor passes through two identical encoder layers. Each layer executes Layer Normalization followed by self-attention computation:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}} + \mathbf{bias}\right)\mathbf{V}$$

where $\mathbf{Q} = \mathbf{x_q W^Q}$, $\mathbf{K} = \mathbf{x_k W^K}$, and $\mathbf{V} = \mathbf{x_v W^V}$. After adding a residual connection and re-normalizing, the tensor enters a feed-forward network comprising a linear layer with GELU activation, dropout, and a final linear projection. Ultimately, the representation of the **[CLS]** token is extracted from the final layer and passed to the classification head, where a Cross-entropy function is applied, in order to extract the logits.

D. Architecture Comparison and Transformer Limitations

While the Transformer has significant representational power, it is severely disadvantaged by the spatial sparsity of the frequency bins in Doppler spectrograms. Transformers lack a strong built-in assumption of "locality," expending computational resources finding correlations across the empty black space within the sparse data. Consequently, it memorizes the training data perfectly (close to 100% training accuracy) but completely collapses on the validation set of strictly unseen subjects due to structural overfitting.

E. Task-Dependent Classification Head

For the Activity Recognition task, the shared backbone outputs to a standard linear classification head trained with Cross-Entropy Loss to classify 5 distinct activities. For the Person Identification task, the problem is formulated through Contrastive Learning. The backbone outputs to a 128-dimensional embedding space supervised by Triplet Margin Loss. At inference time, a K-Nearest Neighbors ($k = 5$) classifier evaluates the unseen subject embeddings.

VI. Experiments and Results

A. Hardware and Execution Setup

The experiments and final testing were executed on a dedicated Hyperstack cloud virtual machine equipped with an NVIDIA L40 GPU featuring 48 GB of VRAM. This substantial memory footprint permitted high-throughput parallel training across all configurations. Real-time telemetry, model checkpointing, and comprehensive training/validation metrics were continuously logged and monitored using the Weights & Biases (W&B) platform. To ensure strict isolation, the test sets consisted purely of completely unseen subjects, testing true model generalization rather than artificial memorization.

Detailed instructions for reproducing these environments and invoking the execution scripts can be found in Appendix B.

B. Performance Analysis

The Micro-ResNet ('ResNet8') proved to be the optimal architecture, consistently outperforming the Transformer and Inception models.

TABLE I: Strict-Isolation Test Metrics (L40 GPU)

Task	Architecture	Accuracy	Status
Activity	ResNet8	85.78%	Converged
Activity	Inception	83.58%	Converged
Activity	Transformer	62.94%	Overfitted
Person ID	ResNet8	83.45%	Converged
Person ID	Inception	80.50%	Converged
Person ID	Transformer	69.35%	Overfitted

The Transformer severely overfitted the training set but failed to generalize. In contrast, ResNet8's 2D CNN inductive biases mapped perfectly to the Doppler spectrograms, effectively extracting localized frequency-time signatures.

All confusion matrices and training/validation loss graphs for the Activity Recognition and Person Identification tasks are provided in the Appendix.

VII. Concluding Remarks

In this project, we demonstrated that massively deep architectures like Transformers and standard ResNet models are not always the optimal choice, especially for sparse datasets like Doppler spectrograms. By scaling down to a Micro-ResNet (ResNet8), we achieved 85.78% accuracy for Activity Recognition and 83.45% for Person Identification on completely unseen subjects.

Future work involves real-time optimization of the ResNet8 model for edge devices, and potentially investigating other forms of data augmentation to further combat sparsity-induced overfitting.

Appendix A
Supplementary Training Logs and Metrics

This appendix provides the training and validation loss/accuracy curves for the best performing model (ResNet8) on both tasks, as well as the complete set of confusion matrices for all architectures across both tasks.

A. Training and Validation Performance

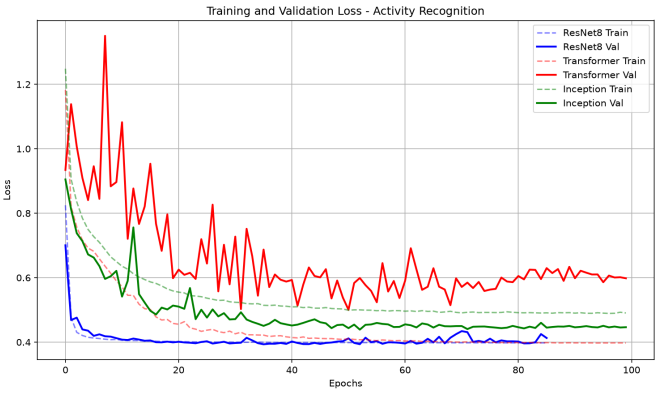


Fig. 6: Combined Training and Validation Loss for Activity Recognition (Hyperstack Runs).

TABLE II: Comprehensive Train, Val, and Test Metrics

Task	Architecture	Train Acc	Val Acc	Test Acc	Status
Activity	ResNet8	100.0%	98.7%	85.8%	Converged
Activity	Inception	99.0%	99.9%	83.6%	Converged
Activity	Transformer	99.9%	91.9%	62.9%	Overfitted
Person ID	ResNet8	92.8%	91.7%	83.5%	Converged
Person ID	Inception	92.3%	86.4%	80.5%	Converged
Person ID	Transformer	93.1%	85.2%	69.4%	Overfitted

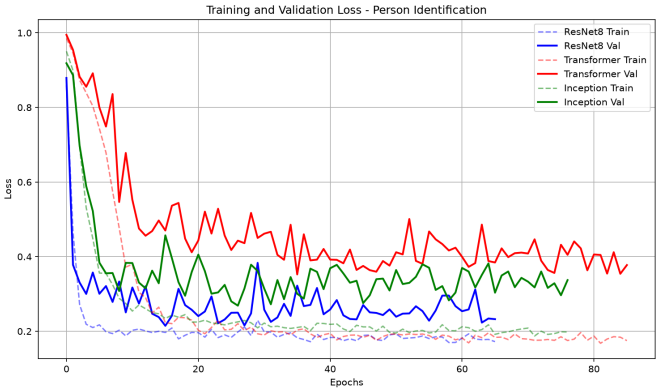
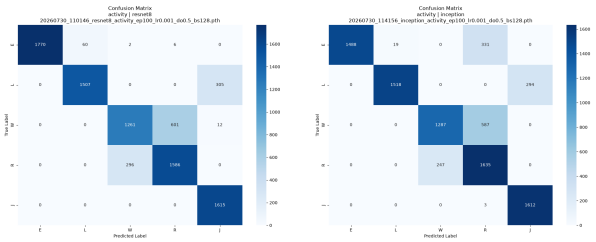
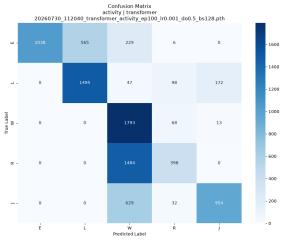


Fig. 7: Combined Training and Validation Loss for Person Identification (Hyperstack Runs).



(a) ResNet8 (b) Inception



(c) Transformer

Fig. 8: Confusion Matrices for Activity Recognition.

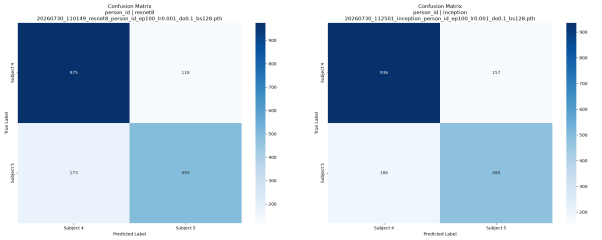
B. Confusion Matrices

Appendix B
Codebase and Execution Guide

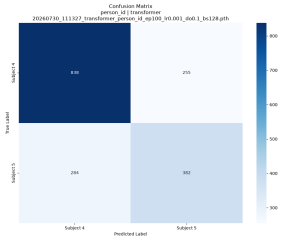
The project repository is structured to provide a robust, unified interface for local and remote execution via a central **`Makefile`**. The codebase supports three distinct paradigms for running the training and testing pipelines, seamlessly adapting the user's configurations across execution environments.

A. Execution Environments

1) **Local Bare Metal:** Direct execution natively in the terminal using the local virtual environment (e.g., **`make train`**). Ideal for quick debugging and syntax checks.
2) **Local Docker:** Wraps the execution within an isolated local container (e.g., **`make train-local`**). Ensures dependency consistency before remote deployment.
3) **Hyperstack Cloud Docker:** Uses the **`remote_exec/deploy.py`** script to securely



(a) ResNet8 (Person ID) (b) Inception (Person ID)



(c) Transformer (Person ID)

Fig. 9: Confusion Matrices for Person Identification.

connect, synchronize the local codebase to the cloud via **`rsync`**, and dynamically provision or resume a GPU cloud instance via the Hyperstack API before interactively executing training inside a container while streaming the logs back (e.g., **`make train-hyperstack`**).

B. Key Scripts and Configurations

The fundamental logic resides in independent, modular scripts, which the `Makefile` coordinates:

- `train.py`: The primary training pipeline. It dynamically parses arguments for architecture (`resnet8`, `inception`, `transformer`), task (`activity`, `person_id`), learning rate, and epochs.
- `test.py`: Evaluates stored weights against the isolated test dataset, producing analytical confusion matrices and accuracy metrics.
- `check_wandb.py` & `download_wandb_history.py`: Integration scripts that query the Weights & Biases API to retrieve performance history and compile comparative results and graphs.

All Makefile commands accept environmental variable overrides for rapid prototyping. For instance, testing a Transformer on the Activity Recognition task locally can be launched as simply as:

```
make train ARCH=transformer TASK=activity
           EPOCHS=10 LR=5e-4
```

C. Remote Sync and Cleanup

Remote operations automatically synchronize the latest codebase changes prior to execution. Following execution, results and serialized weights (`.pt` files) are retrieved using the provided fetch commands:

- **`make pull-hyperstack`**: Retrieves all trained weights from the cloud.
- **`make pull-results-hyperstack`**: Retrieves just the test logs, CSVs, and generated graphs.
- **`make clean-all`**: Safely tears down local containers, remote containers, and halts cloud instances to prevent inactive billing.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NIPS)*, 2017.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

[3] S.-U. Kim and J.-Y. Kim, "Improving human activity recognition through 1d-resnet: A wearable wristband for 14 workout movements," *Processes*, vol. 13, no. 1, p. 207, 2025.

[4] M. Ronald, A. Poulose, and D. S. Han, "isplinception: An inception-resnet deep learning architecture for human activity recognition," *IEEE Access*, vol. 9, pp. 68985–69001, 2021.

[5] F. Meneghello, D. Garlisi, N. Dal Fabbro, I. Tinnirello, and M. Rossi, "Sharp: Environment and person independent activity recognition with commodity ieee 802.11 access points," *IEEE Transactions on Mobile Computing*, vol. 22, pp. 6160–6175, Oct. 2023.

[6] F. Meneghello, D. Garlisi, N. Dal Fabbro, I. Tinnirello, and M. Rossi, "A csi dataset for wireless human sensing on 80 mhz wi-fi channels," *IEEE Communications Magazine*, vol. 61, pp. 146–152, Sept. 2023.

[7] H. Abdelnasser, K. A. Harras, and M. Youssef, "Ubibreathe: A ubiquitous non-invasive wifi-based breathing estimator," in *16th ACM International Symposium on Mobile Ad Hoc Networking and Computing (MobiHoc)*, pp. 277–286, 2015.

[8] Y. Li and L. Wang, "Human activity recognition based on residual network and bilstm," *Sensors (Basel)*, vol. 22, no. 2, p. 635, 2022.

[9] J. Zhang, S. Qiao, Z. Lin, and Y. Zhou, "Human activity recognition based on residual network," *IOP Conference Series: Earth and Environmental Science*, vol. 693, 2021.